# Compte rendu SAE 21

| interface | mot de passe |
|---|---|
| line console | test |
| enable secret | test |
| line vty | test |

## 1- Matériaux

Les matériaux utilisés pour chaque Lan sont:

- Deux routeurs pour chaque Lan (un pour sortir du Lan et un autre qui sera nôtre FAI)

- Fibre optique au coeur du réseau entre les routeurs/câble série ( fibre monomode dans le cas ou nos FAI serait distant de plusieurs kilomètres et pour pouvoir utiliser au mieux les services de téléphonie IP ou bien de la multimode pour des FAI distant de moins de 500m)

- Quatre commutateurs de couche 2 de 24 ports pour chaque Lan utilisant la communication duplex intégral (afin de connecter plusieurs périphériques et aussi pour palier au souci de latence de la communication semi-duplex qui est un peu retardé par les méthodes de contrôle d'accès)

- câble UTP terminées par un connecteur RJ45 de 1000Base-T pour relier les postes et serveurs à nos switch ou STP pour éviter la diaphonie au autre interférence et dans un rayon de moins de 100 mètres de câbles.

  Pour les liaisons switch à poste peu importe le câble croisé ou droit utilisé ils (PC) pourront s'adapter en fonction. Dans le cas contraire, nous activons la fonction " **Auto-Mdix** "sur l'interface du commutateur de couche 3 car elle est déjà active sur celui de couche 2. Cette fonction détecte le type de câble connecté au port et configure les interfaces en conséquant.

- Organiser notre réseau afin qu'il y ait une redondance.

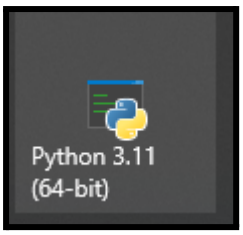
Pour assurer une redondance on utilise le protocol *Spanning tree protocol* (STP) avec du matériel cisco qui gère l'ouverture et la fermeture des différent port ou sinon du *Shortest Path Bridging* (SPB) qui facilite la création et la configuration des réseaux tout en permettant des routages a trajet multiple sinon gérer les port autrement manuellement

Ce sont les Vlan des bâtiments qui devront circuler sur ces liens pour assurer le bon fonctionnement et ils devraient être configurés.

# 2 - plan d'adressage IP

| réseau local du réseau local | Nombre de machines Vlan admin (Vlan10) | Nombre de machines Vlan services (Vlan20) | Nombre de machines Vlan formation_A (Vlan30) | Nombre de machines Vlan formation_B (Vlan40) | Nombre de machines Vlan enseignants (Vlan100) |
|---|---|---|---|---|---|
| Lan1 | 2 | 31 | 90 | 80 | 20 |
| adresse | 172.16.102.0/30 | 172.16.103.0/26 | 172.16.104.0/25 | 172.16.104.128/25 | 172.16.105.0/25 |
| Plage | **172.16.102.1 -172.16.0.2** | **172.16.103.1 - 172.16.103.62** | **172.16.104.1 - 172.16.1.126** | **172.16.104.129 - 172.16.1.218** | **172.16.105.1 - 172.16.105.20** |
| Masque | 255.255.255.252 | 255.255.255.192 | 255.255.255.128 | 255.255.255.128 | 255.255.255.128 |
| | | | | | |
| Lan2 | 8 | 20 | 300 | 300 | 40 |
| adresse | 172.16.64.0/21 | 172.16.72.0/21 | 172.16.80.0/21 | 172.16.88.0/23 | 172.16.90.0/23 |
| Plage | **172.16.64.1 - 172.16.71.254** | **172.16.72.1 - 172.16.79.254** | **172.16.80.61 - 172.16.87.254** | **172.16.88.1 - 172.16.89.46** | **172.16.90.1 - 172.16.90.40** |
| Masque | 255.255.248.0 | 255.255.248.0 | 255.255.248.0 | 255.255.254.0 | 255.255.254.0 |
| | | | | | |
| Lan3 | 8 | 64 | 512 | 514 | 127 |
| adresse | 172.16.0.0/21 | 172.16.8.0/21 | 172.16.16.0/21 | 172.16.24.0/21 | 172.16.32.0/21 |
| Plage | **172.16.0.1 - 172.16.7.254** | **172.16.8.1 - 172.16.15.254** | **172.16.16.1 - 172.16.23.254** | **172.16.24.1 - 172.16.31.254** | **172.16.32.1 - 172.16.33.62** |
| Masque | 255.255.248.0 | 255.255.248.0 | 255.255.248.0 | 255.255.248.0 | 255.255.248.0 |
| | | | | | |
| Lan4 | 20 | 32 | 255 | 512 | 40 |
| adresse | 172.16.92.0/23 | 172.16.94.0/23 | 172.16.96.0/23 | 172.16.98.0/23 | 172.16.100.0/23 |
| Plage | **172.16.91.1 - 172.16.93.254** | **172.16.94.1 - 172.16.95.254** | **172.16.96.1 - 172.16.96.255** | **172.16.98.1 - 172.16.99.254** | **172.16.100.1 - 172.16.100.205** |
| Masque | 255.255.254.0 | 255.255.254.0 | 255.255.254.0 | 255.255.254.0 | 255.255.254.0 |

pour pouvoir vérifier qu'il n'y ai aucun chevauchement depuis mon terminal j'y ai importé la bibliothèque "**Ipaddress**" de python




Ci-dessous une des différentes adresses sous réseau des différents Lan

```
>>> Lan1
(IPv4Network('172.16.102.0/30'), IPv4Network('172.16.103.0/26'), IPv4Network('172.16.104.0/25'), IPv4Network('172.16.104.128/25'), IPv4Network('172.16.105.0/25'))
>>> Lan2
(IPv4Network('172.16.64.0/21'), IPv4Network('172.16.72.0/21'), IPv4Network('172.16.80.0/21'), IPv4Network('172.16.88.0/23'), IPv4Network('172.16.90.0/23'))
>>> Lan3
(IPv4Network('172.16.92.0/23'), IPv4Network('172.16.94.0/23'), IPv4Network('172.16.96.0/23'), IPv4Network('172.16.98.0/23'), IPv4Network('172.16.100.0/23'))
>>> Lan4
(IPv4Network('172.16.102.0/30'), IPv4Network('172.16.103.0/26'), IPv4Network('172.16.104.0/25'), IPv4Network('172.16.104.128/25'), IPv4Network('172.16.105.0/25'))
>>>
```

Grâce à la bibliothèque directement importé il m'a simplement suffit de renseigner les différents sous réseaux par exemple:

**vlan1 = ipaddress.IPv4Network('172.16.102.0/29')**

**vlan2 = ipaddress.IPv4Network('172.16.103.0/26')**

une fois les sous réseaux enregistré tester si il y a un chevauchement:

**vlan1.overlaps(vlan2)**

**Si la valeur false est renvoyé après avoir entré les deux adresses réseaux cela veut simplement dire qu'il n'y a aucun chevauchement dans le cas contraire avec la valeur true nous avons un chevauchement.**

**NB:** Les numéros des vlan ci-dessous sont choisis arbitrairement et ne correspondent pas à celui du sujet.

## Vérification des chevauchement

```
Python 3.11 (64-bit)
>>> import ipaddress
>>> vlan1 = ipaddress.IPv4Network('172.16.102.0/30')
>>> vlan2 = ipaddress.IPv4Network('172.16.103.0/26')
>>> vlan3 = ipaddress.IPv4Network('172.16.104.0/25')
>>> vlan4 = ipaddress.IPv4Network('172.16.104.128/25')
>>> vlan5 = ipaddress.IPv4Network('172.16.105.0/25')
>>> vlan1.overlaps(vlan2)
False
>>> vlan1.overlaps(vlan3)
False
>>> vlan1.overlaps(vlan4)
False
>>> vlan1.overlaps(vlan5)
False
>>> vlan2.overlaps(vlan3)
False
>>> vlan2.overlaps(vlan4)
False
>>> vlan2.overlaps(vlan5)
False
>>> vlan3.overlaps(vlan4)
False
>>> vlan3.overlaps(vlan5)
False
>>> vlan4.overlaps(vlan5)
False
>>>
```

**Lan 1**

```
Python 3.11 (64-bit)
>>> import ipaddress
>>> vlan6 = ipaddress.IPv4Network('172.16.64.0/21')
>>> vlan7 = ipaddress.IPv4Network('172.16.72.0/21')
>>> vlan8 = ipaddress.IPv4Network('172.16.80.0/21')
>>> vlan9 = ipaddress.IPv4Network('172.16.88.0/23')
>>> vlan10 = ipaddress.IPv4Network('172.16.90.0/23')
>>> vlan6.overlaps(vlan7)
False
>>> vlan6.overlaps(vlan8)
False
>>> vlan6.overlaps(vlan9)
False
>>> vlan6.overlaps(vlan10)
False
>>> vlan7.overlaps(vlan8)
False
>>> vlan7.overlaps(vlan9)
False
>>> vlan7.overlaps(vlan10)
False
>>> vlan8.overlaps(vlan9)
False
>>> vlan8.overlaps(vlan10)
False
>>> vlan9.overlaps(vlan10)
False
>>>
```

**Lan 2**

```
Python 3.11 (64-bit)
>>> import ipaddress
>>> vlan11 = ipaddress.IPv4Network('172.16.92.0/23')
>>> vlan12 = ipaddress.IPv4Network('172.16.94.0/23')
>>> vlan13 = ipaddress.IPv4Network('172.16.96.0/23')
>>> vlan14 = ipaddress.IPv4Network('172.16.98.0/23')
>>> vlan15 = ipaddress.IPv4Network('172.16.100.0/23')
>>> Lan3 = vlan11, vlan12, vlan13, vlan14, vlan15
>>> vlan11.overlaps(vlan12)
False
>>> vlan11.overlaps(vlan13)
False
>>> vlan11.overlaps(vlan14)
False
>>> vlan11.overlaps(vlan15)
False
>>> vlan12.overlaps(vlan13)
False
>>> vlan12.overlaps(vlan14)
False
>>> vlan12.overlaps(vlan15)
False
>>> vlan13.overlaps(vlan14)
False
>>> vlan13.overlaps(vlan15)
False
>>> vlan14.overlaps(vlan15)
False
>>>
```

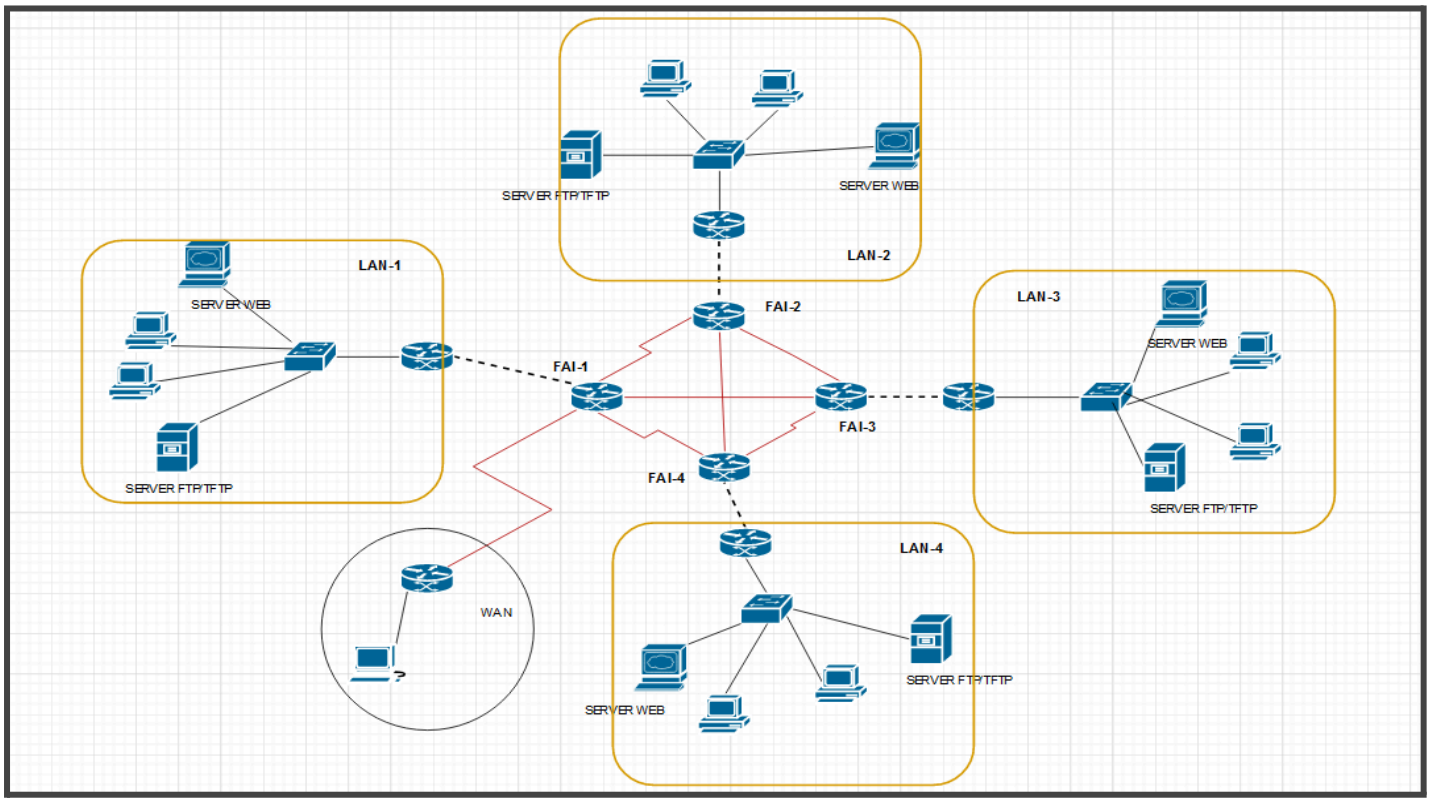
**Lan 3**

```
Python 3.11 (64-bit)
>>> import ipaddress
>>> vlan16 = ipaddress.IPv4Network('172.16.102.0/30')
>>> vlan17 = ipaddress.IPv4Network('172.16.103.0/26')
>>> vlan18 = ipaddress.IPv4Network('172.16.104.0/25')
>>> vlan19 = ipaddress.IPv4Network('172.16.104.128/25')
>>> vlan20 = ipaddress.IPv4Network('172.16.105.0/25')
>>> Lan4 = vlan16, vlan17, vlan18, vlan19, vlan20
>>> vlan16.overlaps(vlan17)
False
>>> vlan16.overlaps(vlan18)
False
>>> vlan16.overlaps(vlan19)
False
>>> vlan16.overlaps(vlan20)
False
>>> vlan17.overlaps(vlan18)
False
>>> vlan17.overlaps(vlan19)
False
>>> vlan17.overlaps(vlan20)
False
>>> vlan18.overlaps(vlan20)
False
>>> vlan18.overlaps(vlan19)
False
>>> vlan20.overlaps(vlan19)
False
>>>
```

**Lan 4**

D'après ces différentes captures nous constatons qu'il n'y a aucun chevauchement entre les différents sous réseaux de nos différents Lan. Il m'est également possible de calculer les plages de ces adresses réseaux à partir de la bibliothèque

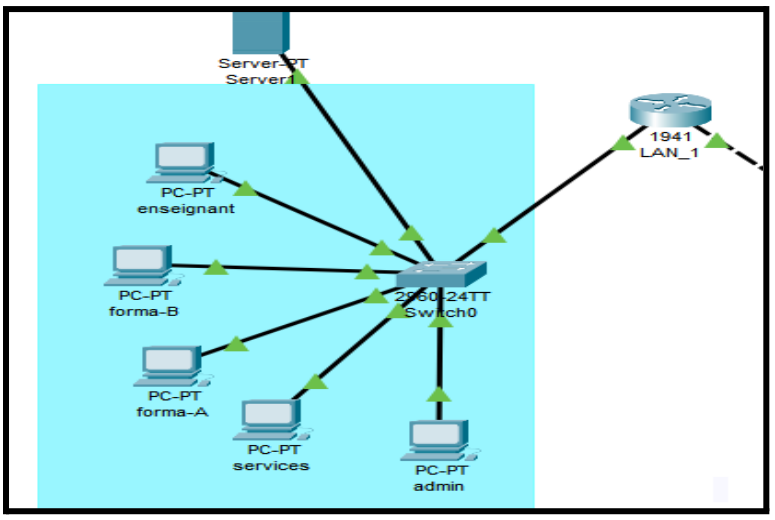
# 3- Topologie




**Topologie maillé.**

# 4- Analyse Packet Trace

## *Création des LAN & VLAN*




```
interface FastEthernet0/1
 switchport trunk native vlan 30
 switchport trunk allowed vlan 10,20,30,40,100
 switchport mode trunk
!
interface FastEthernet0/2
 switchport access vlan 10
 switchport mode access
!
interface FastEthernet0/3
 switchport access vlan 20
 switchport mode access
!
interface FastEthernet0/4
 switchport access vlan 100
 switchport mode access
!
interface FastEthernet0/5
 switchport access vlan 30
 switchport mode access
!
interface FastEthernet0/6
 switchport access vlan 40
 switchport mode access
!
interface FastEthernet0/7
!
interface FastEthernet0/8
!
interface FastEthernet0/9
!
interface FastEthernet0/10
 switchport access vlan 10
 switchport mode access
```

La capture présente à droite illustre la configuration d'un commutateur, cette configuration est la même dans tous les LAN, c'est-à-dire que dans chaque LAN j'ai créé différent vlan dont cinq, numérotés successivement 10, 20, 30, 40, 100. Chaque périphérique final est connecté au switch en **mode access**, avec le numéro de vlan auquel il correspond. C'est fait ainsi pour séparer les groupes qui ont chacun des fonctions distinctes. Mais tous ces vlan pour sortir du réseau ou encore pour communiquer entre eux passeront par le port fastEthernet0/1 en **mode trunk** qui sert de passerelle entre eux.

## *Accès en SSH*

```
LanS1(config)#ip domain-name lan.rt
LanS1(config)#crypto key generate rsa
The name for the keys will be: LanS1.lan.rt
Choose the size of the key modulus in the range of 360 to 4096 for
  General Purpose Keys. Choosing a key modulus greater than 512 ma
  a few minutes.

How many bits in the modulus [512]: 512
% Generating 512 bit RSA keys, keys will be non-exportable...[OK]
```

Ici il fallait tout d'abord créé un nom de domaine arbitrairement choisie **lan.rt**, pour pouvoir générer par la suite une clé d'encryptage, avec la commande **Crypto key generate rsa,** ensuite j'ai choisie une clé de chiffrement de 512 bit qui était le choix par défaut.

```
LanS1(config)#ip ssh version 2
*Mar 1 0:53:13.241: RSA key size needs to be at least 768 bits for ssh version 2
*Mar 1 0:53:13.246: %SSH-5-ENABLED: SSH 1.5 has been enabled
Please create RSA keys (of at least 768 bits size) to enable SSH v2.
```

Ensuite j'ai créé un accès ssh d'où j'ai du faire la commande **ip ssh version 2** qui est conseiller pour pallier au défaut de sécurité de la première version, mais en le faisant comme indiquer sur la capture à la dernière ligne il fallait que je change ma clé de 512 bit par une de 768 bit qui est recommandée pour cette version.

#ip ssh time-out 90 ( Un timeout de 90 secondes est ajouté en cas d'inactivité durant l'authentification.)
#ip ssh authentication-retries 3 (trois essais pour la connexion au switch. Suite à ces essais, la connexion est fermée.)

Afin de rendre incompréhensible tous les mots de passe visible dans la configuration j'ai utilisé cette commande:

```
LanS1(config)#service password-encryption
LanS1(config)#
```

La mise en place du service ssh a été appliquée à tous les nœuds intermédiaire du réseau.

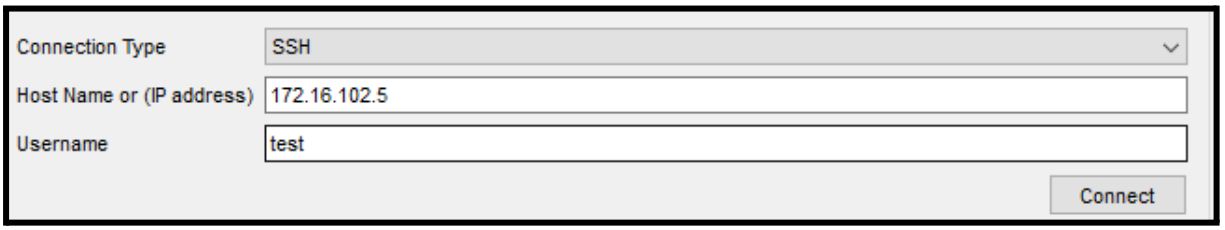
```
interface Vlan10
 ip address 172.16.102.5 255.255.255.248
```

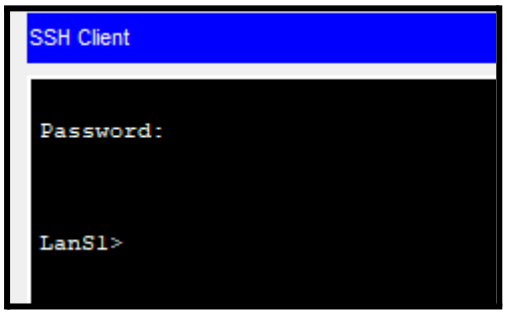
Bien avant de faire un test il faut donner une adresse IP au switch/routeur, sur la capture il s'agit de celle du vlan 10 présent sur le premier LAN (1), et configurer les mots de passe console et secret.

```
LanS1(config)#username test password test
*Mar 1 0:53:54.295: %SSH-5-ENABLED: SSH 1.99 has been enabled
LanS1(config)#line vty 0 15
LanS1(config-line)#login local
LanS1(config-line)#transport input ssh
LanS1(config-line)#
```

Pour tester si la configuration du SSH a été correctement faite j'ai créé un utilisateur nommé **test** avec comme mdp **test**

Les captures ci-dessous prouvent bien que la connexion en ssh depuis le poste du vlan 10 (Administration) fonctionne.




## ***Affectation des différentes adresses***

Lien [github]() vers mon plan d'adressage avec les différents ports en trunk & access.

## ***Routage inter-vlan***

```
interface GigabitEthernet0/0.1
 encapsulation dot1Q 2 native
 ip address 172.16.102.1 255.255.255.248
!
interface GigabitEthernet0/0.2
 encapsulation dot1Q 3
 ip address 172.16.103.1 255.255.255.192
!
interface GigabitEthernet0/0.3
 encapsulation dot1Q 4
 ip address 172.16.104.1 255.255.255.128
!
interface GigabitEthernet0/0.4
 encapsulation dot1Q 5
 ip address 172.16.104.129 255.255.255.128
!
interface GigabitEthernet0/0.5
 encapsulation dot1Q 6
 ip address 172.16.105.1 255.255.255.128
```

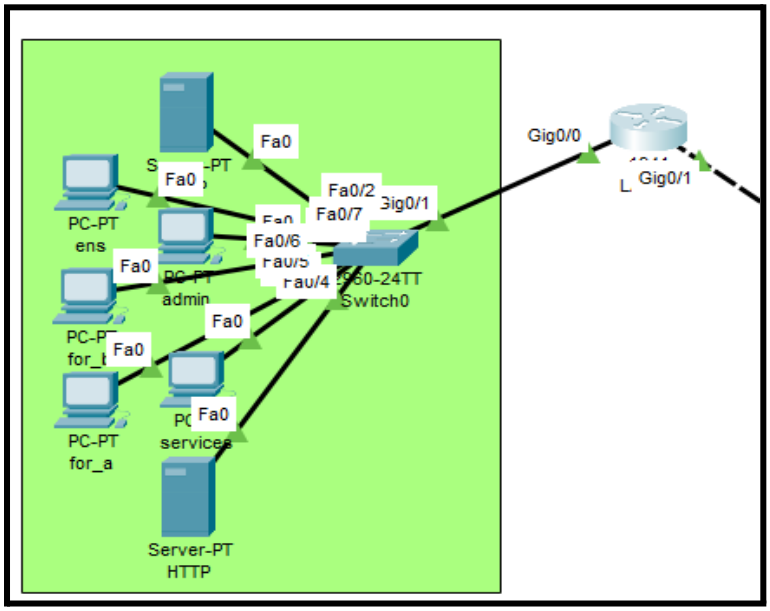
Pour mettre en place le routage inter vlan il m'a fallu créer des sous interfaces car nous disposons de plusieurs sous réseaux, ensuite faire une encapsulation dot1q qui représente la norme IEEE 802.1 et pour chaque encapsulation affecté l'adresse de passerelle des différents des différents réseau sans oublié notifié le vlan native. Le numéro suivant dot1q représente le numéro de vlan.

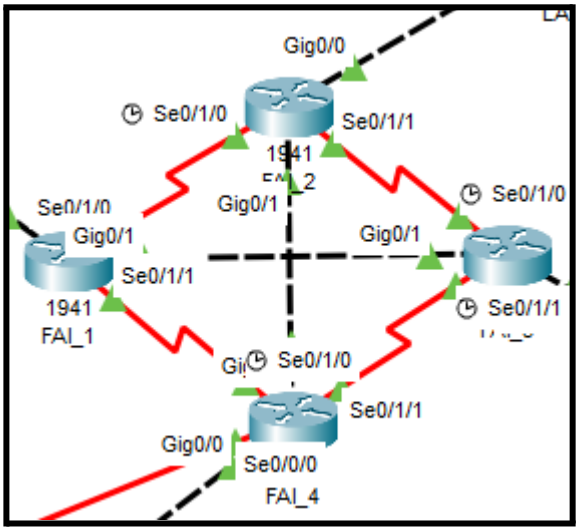
```
Pinging 172.16.105.2 with 32 bytes of data:

Reply from 172.16.105.2: bytes=32 time<1ms TTL=127
Reply from 172.16.105.2: bytes=32 time<1ms TTL=127
Reply from 172.16.105.2: bytes=32 time=11ms TTL=127
Reply from 172.16.105.2: bytes=32 time<1ms TTL=127
```

[liens vidéo inter-vlan](#) du LAN 1

## ***Routage inter site***

Pour mettre en place une connexion entre le LAN 1 à 4, j'ai utilisé un protocol de routage Dynamique qui je trouve est bien plus facile que l'utilisation de route statique il s'agit ici du protocol RIP version 2 pour une mise à jour automatique de la table de routage.




**Lan_1** **Coeur de réseau FAI**

Pour configurer le RIP j'ai utilisé l'adresse de réseau 192.168.0.0 que j'ai découpée en plusieurs sous réseaux pour cela j'ai utilisé un masque en /30 pour attribuer deux adresses utilisables dans un sous réseau.

```
!
interface GigabitEthernet0/1
 ip address 192.168.1.1 255.255.255.252
 duplex auto
 speed auto
```

j'ai affecté l'adresse 192.168.1.1 sur l'interface Gig0/1 qui sort sur le cœur de réseau, ensuite j'ai configuré le

routage dynamique avec les commandes ci-dessous.

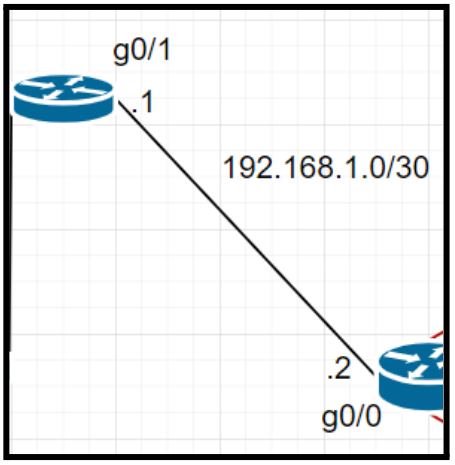
**Lan_1(config)#router rip**
**Lan_1(config-router)#version 2**
**Lan_1(config-router)#network 172.16.0.0**
**Lan_1(config-router)#network 192.168.0.0**

pour que le RIP fonctionne il faut veiller à annoncer  les différents réseaux qui auront accès sinon cela ne marchera pas donc l'adresse réseau du Lan ainsi que l'adresse réseau des interfaces

```
router rip
 version 2
 network 172.16.0.0
 network 192.168.1.0
 no auto-summary
```

et pour connecter tous mes Lan entre eux j'ai dû appliquer la même procédure sur les autres Lan ainsi que sur les routeurs FAI.

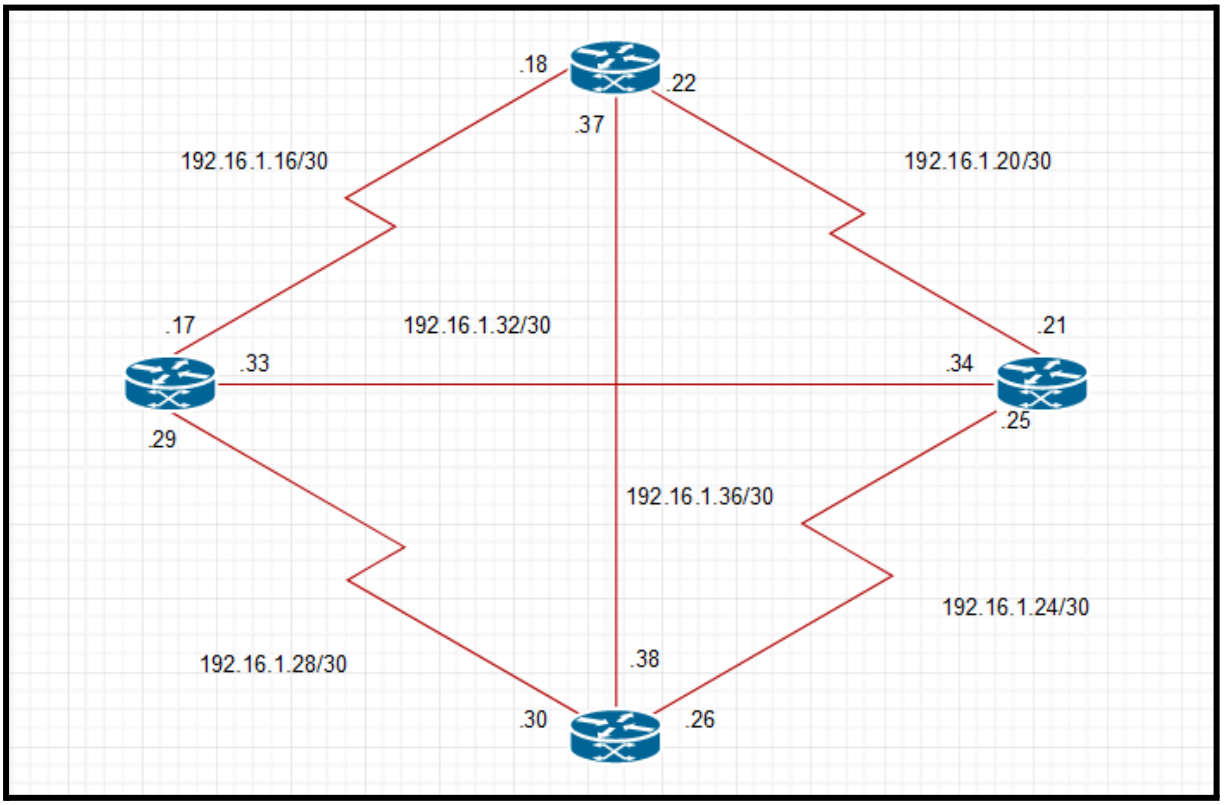
```
interface GigabitEthernet0/0
 ip address 192.168.1.2 255.255.255.252
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 ip address 192.168.9.2 255.255.255.252
 duplex auto
 speed auto
!
interface Serial0/0/0
 no ip address
 clock rate 2000000
 shutdown
!
interface Serial0/0/1
 no ip address
 clock rate 2000000
 shutdown
!
interface Serial0/1/0
 ip address 192.168.2.1 255.255.255.252
!
interface Serial0/1/1
 ip address 192.168.8.2 255.255.255.252
!
```




**adresse interface Lan1**

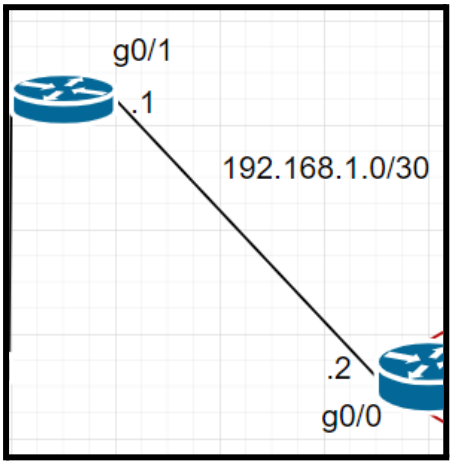
Ici nous avons l'adresse de l'interface directement reliée au Lan 1 ainsi que les interfaces reliées au autre routeur FAI.

## Adresses des différentes interfaces connecté




Comme pour le routeur du Lan 1, il m'a aussi fallu indiquer l'adresse de tous les réseaux qui sont connectés sur ce routeur. [liens vidéo démonstration de ping]

## ***Mode de communication***




```
interface GigabitEthernet0/0
 ip address 192.168.0.1 255.255.255.252
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
```

Concernant les liens entre les différents routeurs à la base j'aurais voulu configurer les interfaces en mode duplex intégral. Mais dans la configuration initiale c'est le mode duplex auto qui était déjà en place.

Il existe deux modes de communication qui sont le duplex intégral et le semi duplex.

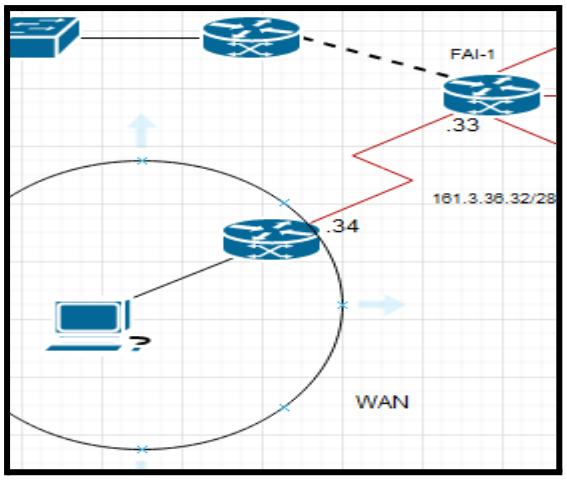
- Duplex intégral: La communication duplex intégrale est un mode de communication où les deux nœuds (routeur) peuvent envoyer et recevoir des données simultanément, sans interruption ni attente. Cela signifie que les canaux de communication sont ouverts dans les deux sens en même temps, permettant une transmission bidirectionnelle.

- Semi-duplex: la communication semi-duplex est un mode de communication où les deux nœuds (routeur) peuvent envoyer et recevoir des données, mais pas simultanément. Dans ce mode, les participants doivent alterner entre l'envoi et la réception des informations.

Dans la configuration packet tracer c'est duplex auto qui est configuré par défaut.  C'est une fonctionnalité dans les dispositifs réseau qui permet de détecter et de négocier automatiquement le mode de duplex approprié pour établir une communication bidirectionnelle efficace.

## Translation d'adresse Nat

Il existe deux types de translation d'adresse sur cette topologie une dynamique et une autre statique.




Sur le routeur FAI-1 qui est relié au WAN et au LAN 1  j'ai choisi comme indiqué dans le cahier des charges de mettre en place le NAT dynamique pour que tous les membres du réseau 172.16.0.0  qui est un réseau privée, puissent sortir sur le wan avec une adresse public et le NAT statique pour les serveurs web accèdent avec une unique adresses publiques.

Tout d'abord l'interface FAI relié au WAN est configuré en "outside" avec la commande "**ip nat outside**" ensuite les trois autres interfaces connecté au trois autres FAI, ainsi que l'interface relié au routeur du LAN 1 en "inside" avec la commande "**ip nat inside**"

```
Router(config)#interface Serial0/0/1
Router(config-if)#ip nat outside
Router(config-if)#int g0/0
Router(config-if)#ip nat inside
Router(config-if)#interface Serial0/0/0
Router(config-if)#ip nat inside
Router(config-if)#interface Serial0/1/0
Router(config-if)#ip nat inside
Router(config-if)#interface Serial0/1/1
Router(config-if)#ip nat inside
Router(config-if)#ex
```

Ensuite est configurée la liste d'accès qui servira plus tard à faire le lien entre les adresses privées et les adresses publiques ci-dessous.

```
Router(config)#access-list 99 permit 172.16.0.0 0.0.255.255
Router(config)#
```

<u>Adresse utilisé pour NAT-PAT:</u> 161.3.36.35 - 161.3.36.42

```
Router(config)#access-list 99 permit 172.16.0.0 0.0.255.255
Router(config)#ip nat pool dynamique 161.3.36.35 161.3.36.42 netmask 255.255.255.240
Router(config)#ip nat inside source list 99 pool dynamique overload
Router(config)#
```

<u>Adresse utilisé pour NAT:</u> 161.3.36.43 - 161.3.36.46

```
Router(config)#ip nat inside source static tcp 172.16.102.4 80 161.3.36.43 80
Router(config)#ip nat inside source static tcp 172.16.64.4 80 161.3.36.44 80
Router(config)#ip nat inside source static tcp 172.16.0.4 80 161.3.36.45 80
Router(config)#ip nat inside source static tcp 172.16.92.4 80 161.3.36.46 80
Router(config)#
```

Les serveurs WEB pourront accéder au WAN avec une adresse statique.

Lorsque nous exécutons la comme “**show ip nat translation**” nous pouvons voir si cela fonctionne

```
Router#sh ip nat translations
Pro  Inside global      Inside local        Outside local       Outside global
icmp 161.3.36.35:16     172.16.105.2:16     8.8.8.1:16          8.8.8.1:16
icmp 161.3.36.35:17     172.16.105.2:17     8.8.8.1:17          8.8.8.1:17
icmp 161.3.36.35:18     172.16.105.2:18     8.8.8.1:18          8.8.8.1:18
icmp 161.3.36.35:19     172.16.105.2:19     8.8.8.1:19          8.8.8.1:19
icmp 161.3.36.35:1      172.16.102.4:1      8.8.8.1:1           8.8.8.1:1
icmp 161.3.36.35:2      172.16.102.4:2      8.8.8.1:2           8.8.8.1:2
icmp 161.3.36.35:3      172.16.102.4:3      8.8.8.1:3           8.8.8.1:3
icmp 161.3.36.35:4      172.16.102.4:4      8.8.8.1:4           8.8.8.1:4
tcp 161.3.36.43:80      172.16.102.4:80     ---                 ---
tcp 161.3.36.44:80      172.16.64.4:80      ---                 ---
tcp 161.3.36.45:80      172.16.0.4:80       ---                 ---
tcp 161.3.36.46:80      172.16.92.4:80      ---                 ---
```

## ***Mise en place des ACL***

```
Vlan admin

ip access-list extended adm
permit ip 172.16.80.0 0.0.7.255 host 172.16.64.3
permit ip 172.16.88.0 0.0.7.255 host 172.16.64.3
permit ip 172.16.72.0 0.0.7.255 host 172.16.64.3
permit ip 172.16.90.0 0.0.7.255 host 172.16.64.3

deny ip 172.16.64.0 0.0.7.255 172.16.80.0 0.0.7.255
deny ip 172.16.64.0 0.0.7.255 172.16.88.0 0.0.7.255
deny ip 172.16.64.0 0.0.7.255 172.16.72.0 0.0.7.255
deny ip 172.16.64.0 0.0.7.255 172.16.90.0 0.0.7.255

int g0/0.1
ip access-group adm out
```

A la demande du cahier des charges il fallait qu' aucun des autres vlan présent dans un Lan ne puissent accéder au vlan administration qui n’est réservé qu’à la configuration des switchs et des routeurs, mais de les laissé pouvoir accéder au serveur TFTP qui est présent dans ce Vlan.
En bleu nous avons permis tous les vlan l’accès au TFTP et en vert refusé l’accès au réseau, ensuite nous avons affecté cette liste d’accès sur la sous interface relié au vlan administration en “OUT” . Normalement une liste d’accès étendu doit être placé au plus proche de la source c’est à dire en “IN” mais dans ce cas là ma liste d’accès ne sera pas fonctionnel

```
permit tcp 172.16.64.0 0.0.7.255 host 172.16.72.3 eq www
permit ip 172.16.64.0 0.0.7.255 host 8.8.8.9
```

Toujours sur la liste d'accès j'ai laissé la possibilité au Vlan administration de pouvoir ping le serveur HTTP du lan ainsi que de pouvoir ping le WAN.

J'ai décidé de ne pas mettre de liste d'accès sur les interface des autres vlan pour leur laissé l'accès au serveur HTTP. Et sur le LAN - 4 si mes ping fonctionnaient correctement dans cette direction je devais laisser l'accès au différent sous réseau qui devait y être affecté

# ***TP SAE21***

## ***Inter-vlan***

Afin de mettre en place le routage inter vlan nous avons utilisé le plan d'adressage d'un camarade, nous avons choisie les adresses de son LAN 2 que nous avons attribué au PC

| LAN 2 | 8 | 20 |
|---|---|---|
| Ad/Masque | 172.16.5.0/28 | 172.16.6.0/27 |
| Plage | 172.16.5.1-172.16.5.14 | 172.16.6.1-172.16.6.30 |

Nous avons ensuite crée deux vlan (10 & 20) qui sont les vlan service et administration

```
te to down
Switch(config)#vlan 10
Switch(config-vlan)#name VLAN10
Switch(config-vlan)#exit
Switch(config)#vlan 20
Switch(config-vlan)#vlan 20
Switch(config-vlan)#name service
Switch(config-vlan)#exit
Switch(config)#
```

```
interface FastEthernet0
!
interface GigabitEthernet1/0/1
 switchport trunk native vlan 20
 switchport trunk allowed vlan 10,20
 switchport mode trunk
!
interface GigabitEthernet1/0/2
 switchport access vlan 10
 switchport mode access
!
interface GigabitEthernet1/0/3
 switchport access vlan 20
 switchport mode access
!
```

Par la suite nous avons configuré les port en access pour chacun des vlan et un lien trunk qui sera connecté au routeur.
Après cette étape nous avons ensuite configuré les sous interfaces pour nos deux passerelles.

```
interface GigabitEthernet0/0/0
 no ip address
 negotiation auto
!
interface GigabitEthernet0/0/0.1
 encapsulation dot1Q 10
 ip address 172.16.5.1 255.255.255.240
!
interface GigabitEthernet0/0/0.2
 encapsulation dot1Q 20 native
 ip address 172.16.6.1 255.255.255.224
!
```

Enfin nous avons pu réaliser nos test ping dans notre réseau

```
C:\Users\admin>ping 172.16.5.2

Envoi d'une requête 'Ping'  172.16.5.2 avec 32 octets de données :
Réponse de 172.16.5.2 : octets=32 temps=1 ms TTL=127
Réponse de 172.16.5.2 : octets=32 temps=3 ms TTL=127
Réponse de 172.16.5.2 : octets=32 temps=2 ms TTL=127
Réponse de 172.16.5.2 : octets=32 temps=1 ms TTL=127

Statistiques Ping pour 172.16.5.2:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 1ms, Maximum = 3ms, Moyenne = 1ms
```

```
C:\Users\admin>ping 172.16.6.2

Envoi d'une requête 'Ping'  172.16.6.2 avec 32 octets de données :
Réponse de 172.16.6.2 : octets=32 temps=2 ms TTL=127
Réponse de 172.16.6.2 : octets=32 temps=2 ms TTL=127
Réponse de 172.16.6.2 : octets=32 temps=3 ms TTL=127
Réponse de 172.16.6.2 : octets=32 temps=3 ms TTL=127

Statistiques Ping pour 172.16.6.2:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 2ms, Maximum = 3ms, Moyenne = 2ms
```

Ping vlan 20 vers 10

Ping vlan 10 vers vlan 20

## ***Inter-site***

Pour mettre en place l'inter-site nous avons découpé 192.168.0.0 en trois sous réseaux de deux adresses chacun pour joindre le LAN 1 et le LAN 3

| LAN 3 | 8 | 64 |
|---|---|---|
| Ad/Masque | 172.16.13.0/28 | 172.16.14.0/25 |
| Plage | 172.16.13.1-172.16.13.14 | 172.16.14.1-172.16.14.126 |

| LAN 1 | 2 | 31 |
|---|---|---|
| Ad/Masque | 172.16.0.0/30 | 172.16.1.0/26 |
| Plage | 172.16.0.1-172.16.0.2 | 172.16.1.1-172.16.1.62 |

Nous avons aussi attribué l'adresse 192.168.0.9 à l'interface connecté au FAI et 192.168.0.10 à l'autre interface et configuré le rip version 2 pour prendre en charge le VLSM.

```
interface GigabitEthernet0/0/1
 ip address 192.168.0.9 255.255.255.252
 negotiation auto
!
```

```
interface GigabitEthernet0/0/0
 ip address 192.168.0.18 255.255.255.252
 negotiation auto
!
interface GigabitEthernet0/0/1
 ip address 192.168.0.10 255.255.255.252
 negotiation auto
!
```

Interface routeur lan

Interface Fai

```
Router(config)#router rip
Router(config-router)#version 2
Router(config-router)#network 192.168.0.0
Router(config-router)#no auto-summary
```

Cette capture montre la configuration du RIP sur le FAI il a juste besoin de connaitre le réseau 192.168.0.0

Mais sur le routeur du LAN nous avons annoncé deux réseaux (192.168.0.0 et 172.16.0.0).

Nous avons aussi configuré notre deuxième interface et lui avons attribué l'adresse 192.168.0.18 qui est connecté à un autre routeur avec l'adresse 192.168.0.17

```
interface GigabitEthernet0/0/0
 ip address 192.168.0.18 255.255.255.252
 negotiation auto
!
```

Sachant que nous avion déjà annoncé les réseaux précédemment nous avons juste à faire un ping pour vérifier

```
C:\Users\admin>ping 172.16.13.2

Envoi d'une requête 'Ping'  172.16.13.2 avec 32 octets de données :
Réponse de 172.16.13.2 : octets=32 temps=1 ms TTL=124
Réponse de 172.16.13.2 : octets=32 temps=2 ms TTL=124
Réponse de 172.16.13.2 : octets=32 temps=2 ms TTL=124
Réponse de 172.16.13.2 : octets=32 temps=1 ms TTL=124

Statistiques Ping pour 172.16.13.2:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 1ms, Maximum = 2ms, Moyenne = 1ms
```

En conclusion avec le RIP nous annonçons les réseaux directement connectés.

# ***Nat***

Pour mettre en place le WAN nous avons configuré un troisième poste avec l'adresse 8.8.8.0/32.

Ensuite sur le routeur connecté WAN, nous avons changé l'adresse de réseau 192.168.0.0 en 161.3.36.32/28 qui est une adresse publique.

L'interface de ce routeur et l'interface du routeur suivant 161.3.36.34

```
interface GigabitEthernet0/0
 ip address 161.3.36.33 255.255.255.240
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 ip address 8.8.8.2 255.255.255.252
 duplex auto
 speed auto
!
```

En faisant le RIP nous avons annoncé le 161.3.36.0, et sur le routeur wan nous avons annoncé

```
router rip
 version 2
 network 8.0.0.0
 network 161.3.0.0
 no auto-summary
!
```

Après nous avons tester le ping cela à fonctionner

```
C:\Users\admin>ping 8.8.8.1

Envoi d'une requête 'Ping'  8.8.8.1 avec 32 octets de données :
Réponse de 8.8.8.1 : octets=32 temps=2 ms TTL=125
Réponse de 8.8.8.1 : octets=32 temps=2 ms TTL=125
Réponse de 8.8.8.1 : octets=32 temps=2 ms TTL=125
Réponse de 8.8.8.1 : octets=32 temps=1 ms TTL=125

Statistiques Ping pour 8.8.8.1:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 1ms, Maximum = 2ms, Moyenne = 1ms
```

Pour que la translation d'adresse nous avons configuré les interfaces pour activer

```
Router(config)#int gi0/0/0
Router(config-if)#ip nat outside
Router(config-if)#int gi0/0/1
Router(config-if)#ip nat inside
```

Après nous avons configuré une liste d'accès pour le réseau 172.16.0.0 et configuré le  pool d'adresse pour la translation.

```
Router(config)#int gi0/0/0
Router(config-if)#ip nat outside
Router(config-if)#int gi0/0/1
Router(config-if)#ip nat inside
Router(config-if)#exit
Router(config)#access-list 1 permit 172.16.0.0 0.0.255.255
Router(config)#ip nat pool sael 161.3.36.35 161.3.36.46
% Incomplete command.

Router(config)#$ sael 161.3.36.35 161.3.36.46 netmask 255.255.255.240
Router(config)#ip nat inside source list 1 pool sael overload
```

Nous avons refait un ping et vérifier la translation avec **show ip nat translation** et grâce au **debug ip nat** nous voyons la translation en temps réel